Graph Neural Networks from the Ground Up

Graphs as data, message passing, the GCN layer, depth and its limits, with runnable code

---

A 20-slide tutorial • companion code: `data.py`, `gcn_numpy.py`, `gnn.py`, `train.py`, `experiments.py`

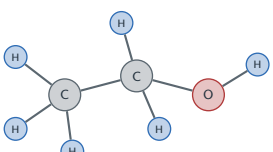
Why graphs? Data that is made of relations

Much data is not a table of independent rows. It is **things plus the relations between them**: atoms joined by bonds, papers joined by citations, people joined by friendships. A **graph** $G = (V, E)$ stores exactly that: a set of **nodes** V and a set of **edges** E between pairs of nodes.

A **graph neural network** (GNN) is a network whose layers let every node combine its own features with those of its neighbours. One small rule, applied at every node.

What the models we already know cannot do

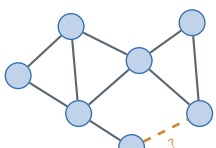
- **MLP**: wants a fixed-length vector per example and treats examples as independent. It has no input for "who is connected to whom".
- **CNN**: wants a regular grid, where every pixel has the same neighbours in the same places (up, down, left, right). A node in a graph has any number of neighbours, in no order.



molecule: atoms, bonds predict a property of the whole graph



citations: papers, "cites" predict the topic of a node



social network: people, ties predict a missing edge

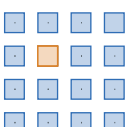
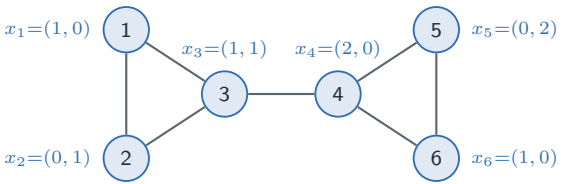


image: pixels on a grid, a very regular graph (the CNN's special case)

Do the relations carry information? In the Cora citation network used later (2,708 papers, 5,278 citation links, 7 topics), 81% of the links join two papers on the *same* topic. A paper's neighbours say a lot about it. Slide 19 measures how much a model gains by using them. Surveys: [17]; origins: [1].

Representing a graph: edge list, A, D and X



The running example: 6 nodes, 7 edges, 2 features per node.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}, \; d = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 3 \\ 2 \\ 2 \end{bmatrix}, \; X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix}$$

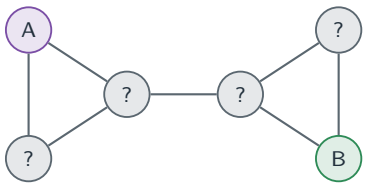
Edge list: $(1,2), (1,3), (2,3), (3,4), (4,5), (4,6), (5,6)$

- **Adjacency matrix** $A \in \{0,1\}^{n \times n}$: $A_{vu} = 1$ if an edge joins v and u. Row v lists the **neighbours** $\mathcal{N}(v)$ of v.
- **Degree** $d_v = \sum_u A_{vu} = |\mathcal{N}(v)|$, $D = \mathrm{diag}(d)$. Check: $\sum_v d_v = 14 = 2|E|$.
- **Node features** $X \in \mathbb{R}^{n \times F}$: row v describes node v (the words of a paper, the type of an atom). No features? Use $X = I$, a one-hot id per node.
- **Undirected**: $A = A^\top$ (friendship, bonds). **Directed**: $A_{vu} \neq A_{uv}$ ("cites", "follows"). We symmetrise citations and work undirected.
- **Self-loop**: an edge from a node to itself, $A_{vv} = 1$. Our data has none; slide 8 adds them on purpose.

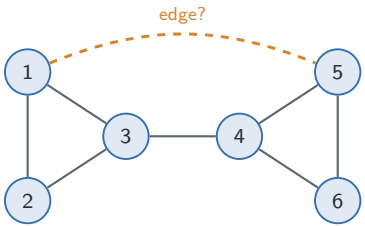
Real graphs are sparse: store the edge list

Cora's A has $2708^2 \approx 7.3$ million entries; only $2 \times 5{,}278$ are non-zero (0.14%). Mean degree 3.9, median 3, maximum 168. The code builds sparse matrices from the edges (`graph_operators`, `gnn.py`); a product AH costs $O(|E|F)$, not $O(n^2F)$.

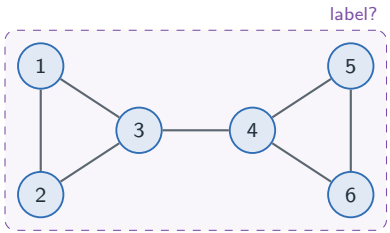
Three kinds of prediction: node, edge, graph



node level: one answer per node



edge level: one answer per node pair



graph level: one answer per graph

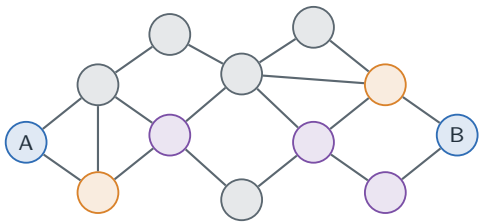
	Node	Edge (link)	Graph
question	what is node v?	are u and v related?	what is this whole graph?
example	topic of a paper; role of a user	friend suggestion	is this molecule toxic?
data	**one** big graph, few labelled nodes	one graph, some edges hidden	**many** small graphs, a label each
output head	$\hat{y}_v = \mathrm{softmax}(h_v W)$	$s_{uv} = \sigma(h_u \cdot h_v)$	$\hat{y}_G = \mathrm{MLP}\left(\sum_{v \in G} h_v\right)$

One encoder, three heads

A GNN turns the features x_v into an **embedding** $h_v \in \mathbb{R}^{F'}$ per node, a summary of the node *and its surroundings*. The three tasks differ only in the last step.

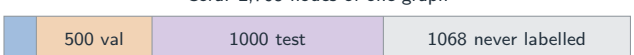
This deck trains node classifiers (slides 5–19) and shows the graph-level machinery on slide 14. Message passing [3] came from graph-level chemistry, GCN [2] from node classification.

Labelled data on a graph: masks instead of separate files



train (label used) validation test unlabelled

Cora: 2,708 nodes of one graph



140 train = 20 per class (5.2% of the nodes)

Our split: for each seed, 20 random nodes per class for training, then 500 validation and 1000 test nodes at random from the rest. These are the *sizes* of the Planetoid split [12], **not** the fixed public split of [2]. Results depend on the split [13]; we report 10.

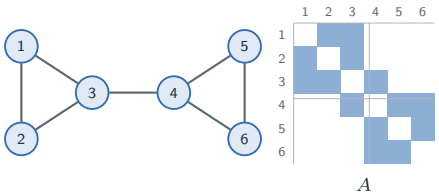
A **label** y_v is the known answer for node v (the topic of a paper). Labels are expensive; edges and features are cheap.

Why not split the rows, as for a table?

With i.i.d. data we cut the table in three and never show test rows to the model. Here the examples are *connected*: remove the test nodes and the training nodes lose neighbours. So we keep **one graph** and split only the **labels**, with three boolean **masks** over the nodes.

- **Transductive** (semi-supervised) learning: the model sees the features and edges of *all* nodes while training, test nodes included. Only their labels are hidden. Validation labels steer early stopping; test labels give one final number.
- The loss averages over the training mask V_L only:
$$\mathcal{L} = -\tfrac{1}{|V_L|}\textstyle\sum_{v\in V_L} \ln p_v[y_v], \qquad p_v = \mathrm{softmax}(z_v).$$
- **Inductive** setting: test on nodes or graphs unseen in training [4] (slide 14).
- **Leakage** = a test *label* reaching the model. `test_gnn.py` flips all non-training labels and checks that training does not change.

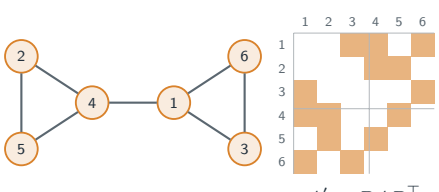
Node order is arbitrary: permutation equivariance and invariance



A

numbering 1

same graph, different matrices



$A' = PAP^\top$

numbering 2

The numbers 1..6 are our bookkeeping, not data. A relabelling is a permutation matrix P: $X \to PX$, $A \to PAP^\top$. There are $6! = 720$ numberings of this graph, about 10^{8121} of Cora.

What we must demand of a model f

Invariance (one output per graph): same answer, $f(PAP^\top, PX) = f(A, X)$.

Equivariance (one output per node): the output rows are reordered like the nodes.

$$F(PAP^\top, PX) = P\,F(A, X)$$

The layer of slide 8 passes, since $P^\top P = I$ and an elementwise σ commutes with P:

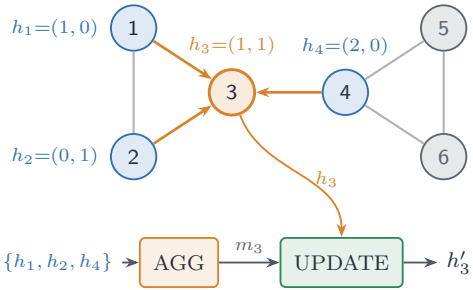
$$\sigma\big((P\hat{A}P^\top)(PX)W\big) = P\,\sigma(\hat{A}XW).$$

Measured on the example (`walkthrough.py`, part 4)

	numbering 1	numbering 2
GCN layer, old node 4	(.211, .202)	(.211, .202)
GCN layer, sum of rows	(1.199, 1.003)	(1.199, 1.003)
linear layer on flat $[A, X]$	2.061	-0.680

$F(PAP^\top, PX)$ and $P\,F(A, X)$ agree exactly.

Message passing: aggregate the neighbours, then update



One message-passing layer [3]

$$h'_v \;=\; \mathrm{UPDATE}\Big(h_v,\; \mathrm{AGG}\big(\{\, h_u : u \in \mathcal{N}(v) \,\}\big)\Big)$$

AGG	at node 3	all nodes at once	cannot tell apart
sum	$(3,\ 1)$	AH	keeps counts (slide 13)
mean	$(1,\ \frac{1}{3})$	$D^{-1}AH$	$\{a,b\}$ and $\{a,a,b,b\}$
max	$(2,\ 1)$	elementwise	$\{a,b\}$ and $\{a,b,b\}$

- Each node holds a vector h_v, starting from its features: $h_v^{(0)} = x_v$.
- **Aggregate**: collect the neighbours' vectors into one **message** m_v. The neighbours form a *set*: no order, any size. So AGG must ignore order and accept any number of inputs: sum, mean, max [9].
- **Update**: combine the message with the node's own vector, using **trainable weights** and a nonlinearity, e.g. $h'_v = \mathrm{ReLU}(h_v W_1 + m_v W_2)$.
- The **same weights at every node**: the number of parameters does not depend on the size of the graph, and a trained layer runs on any graph.

Why this is the right shape

Order-free aggregation plus shared weights gives permutation equivariance (slide 6) by construction. In a CNN the "neighbours" are the pixels under the filter; they have a fixed order, so each gets its own weight. On a graph they have none, so they are pooled first.

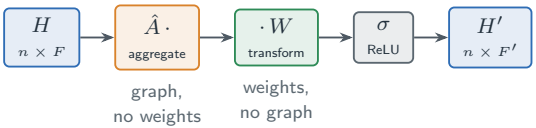
GCN, GraphSAGE, GAT and GIN (slides 8, 12, 13) differ only in AGG and UPDATE.

The GCN layer: one matrix equation

Graph convolutional layer (Kipf & Welling [2])

$$H' = \sigma\big(\hat{A}\, H\, W\big), \qquad \hat{A} = \tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}, \qquad \tilde{A} = A + I$$

$H \in \mathbb{R}^{n\times F}$: one row per node. $W \in \mathbb{R}^{F\times F'}$: the trainable weights. $\tilde{D} = \mathrm{diag}(\tilde{d})$, $\tilde{d}_v = d_v + 1$. σ: ReLU.



The same equation, one node at a time (h_v = row v of H):

$$h'_v = \sigma\Big(\sum_{u\,\in\,\mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{\tilde{d}_v\,\tilde{d}_u}}\; h_u\, W\Big)$$

Message passing with a **weighted sum** as AGG and "times W, then σ" as UPDATE. **Parameters**: $F\cdot F'$ per layer, whatever n is. Our Cora model $1433 \to 16 \to 7$ with biases has $1433\cdot 16 + 16 + 16\cdot 7 + 7 =$ **23,063**.

Why add self-loops? $(AH)_v$ sums the neighbours only: a node's *own* features would vanish from its update. With $\tilde{A} = A + I$ every node is also its own neighbour.

Why normalise? $(\tilde{A}H)_v$ is a sum over $\tilde{d}_v$ nodes. A paper with 168 links would get a vector roughly 40 times longer than a typical one, and every layer multiplies again:

- on the example graph the largest eigenvalue of $\tilde{A}$ is 3.41, so k layers can scale features by 3.41^k;
- all eigenvalues of $\hat{A}$ lie in $(-1,\,1]$. Nothing explodes.

Why $\tilde{D}^{-1/2}\cdots\tilde{D}^{-1/2}$, not the mean $\tilde{D}^{-1}\tilde{A}$?

- The mean weights each neighbour by $1/\tilde{d}_v$. The symmetric form uses $1/\sqrt{\tilde{d}_v\tilde{d}_u}$: a message from a hub u (large $\tilde{d}_u$) counts for less.
- $\hat{A}$ stays **symmetric**: real eigenvalues, orthogonal eigenvectors (slide 11). Both work in practice; GraphSAGE uses the mean (slide 12).

Origin: [2] derives the layer as a first-order approximation of a spectral filter on the graph Laplacian $L = I - D^{-1/2}AD^{-1/2}$. It can be used without that derivation.

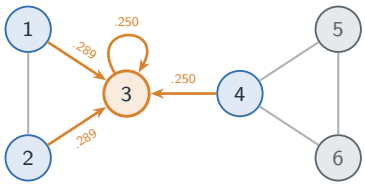
A GCN layer by hand on the example graph

1. **Self-loops and degrees.** $\tilde{A} = A + I, \quad \tilde{d} = (3, 3, 4, 4, 3, 3).$
2. **Normalise.** $\hat{A}_{vu} = 1/\sqrt{\tilde{d}_v \tilde{d}_u}$ wherever $\tilde{A}_{vu} = 1$:

$$\hat{A} = \begin{bmatrix} .333 & .333 & .289 & 0 & 0 & 0 \\ .333 & .333 & .289 & 0 & 0 & 0 \\ \mathbf{.289} & \mathbf{.289} & \mathbf{.250} & \mathbf{.250} & 0 & 0 \\ 0 & 0 & .250 & .250 & .289 & .289 \\ 0 & 0 & 0 & .289 & .333 & .333 \\ 0 & 0 & 0 & .289 & .333 & .333 \end{bmatrix} \qquad \begin{aligned} \tfrac{1}{3} &= \tfrac{1}{\sqrt{3\cdot 3}} \\ .289 &= \tfrac{1}{\sqrt{3\cdot 4}} \\ \tfrac{1}{4} &= \tfrac{1}{\sqrt{4\cdot 4}} \end{aligned}$$

3. **Aggregate**: $S = \hat{A}X$. 4. **Transform**: SW. 5. **ReLU**.

$$\underbrace{\begin{bmatrix} .622 & .622 \\ .622 & .622 \\ \mathbf{1.039} & \mathbf{.539} \\ 1.039 & .827 \\ .911 & .667 \\ .911 & .667 \end{bmatrix}}_{S=\hat{A}X} \underbrace{\begin{bmatrix} 1 & -1 \\ -1 & 1.5 \end{bmatrix}}_{W} = \begin{bmatrix} 0 & .311 \\ 0 & .311 \\ \mathbf{.500} & \mathbf{-.231} \\ .211 & .202 \\ .244 & .089 \\ .244 & .089 \end{bmatrix} \xrightarrow{\text{ReLU}} \underbrace{\begin{bmatrix} 0 & .311 \\ 0 & .311 \\ \mathbf{.500} & \mathbf{0} \\ .211 & .202 \\ .244 & .089 \\ .244 & .089 \end{bmatrix}}_{H'}$$



Row 3 in detail (bold; `walkthrough.py`, part 3)

Node 3 has neighbours 1, 2, 4 and itself:

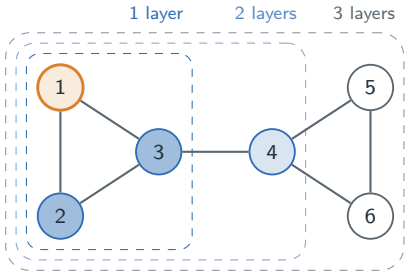
$$\begin{aligned} s_3 &= \tfrac{1}{4}x_3 + \tfrac{1}{\sqrt{12}}x_1 + \tfrac{1}{\sqrt{12}}x_2 + \tfrac{1}{4}x_4 \\ &= \tfrac{1}{4}(1,1) + .289\,(1,0) + .289\,(0,1) + \tfrac{1}{4}(2,0) \\ &= (\mathbf{1.039},\ \mathbf{0.539}) \end{aligned}$$

$$s_3 W = (1.039 - .539,\ -1.039 + 1.5 \cdot .539) = (.500,\ -.231)$$

$$h'_3 = \mathrm{ReLU}(s_3 W) = (\mathbf{0.500},\ \mathbf{0})$$

- Nodes 1 and 2 have the same neighbourhood $\{1, 2, 3\}$, so they get **the same output** although $x_1 \neq x_2$. Likewise 5 and 6.
- With the plain mean $\tilde{D}^{-1}\tilde{A}$, $s_3 = (1,\ 0.5)$; unnormalised, $(4,\ 2)$.

Stacking layers: k layers see k hops

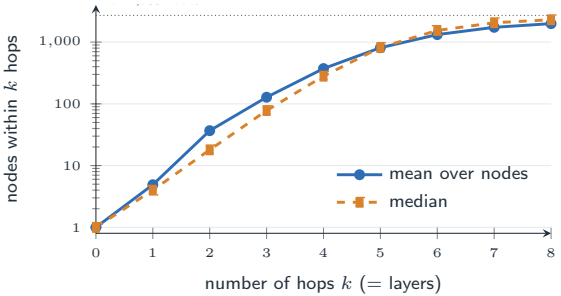


Which inputs can influence node 1 (orange)? Its **receptive field** grows by one hop per layer.

A 2-layer GCN (the model we train):

$$Z = \hat{A}\ \mathrm{ReLU}\big(\hat{A}XW^1\big)\,W^2, \qquad \hat{y}_v = \mathrm{softmax}(z_v)$$

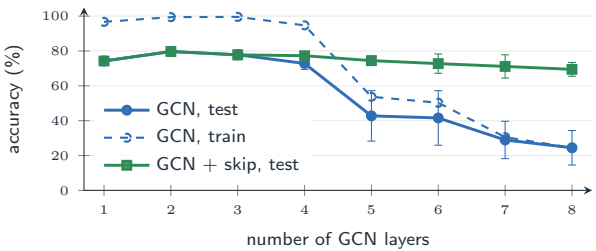
- Layer 1 mixes each node with its neighbours. Layer 2 mixes those mixtures: now neighbours of neighbours contribute. Each layer has its own W^ℓ; $\hat{A}$ is shared.
- In general $h_v^{(k)}$ depends on x_u only if u is within k hops, i.e. $(\tilde{A}^k)_{vu} \neq 0$. Example: $(\hat{A}^2)_{1\cdot} = (.31,\ .31,\ .26,\ .07,\ 0,\ 0)$: node 4 has arrived, nodes 5, 6 not yet.



Measured on Cora by `experiments.py` (the node itself counts).

- **Neighbourhoods grow fast.** On Cora a node reaches 5 nodes on average in 1 hop, 37 in 2, 373 in 4 (14% of the graph) and 74% in 8.
- So 2 layers already pool information from dozens of papers. That is why shallow GNNs work, unlike CNNs or transformers where depth keeps paying.
- **The cost of depth**: with many layers every node's receptive field is nearly the whole graph, and all nodes start to compute the same average. Next slide.

Depth has a price: over-smoothing



Measured: Cora, mean ± std over 5 splits; train = the 140 labelled nodes.

- **Two layers are best**: 74.3% with 1 layer, **79.7%** with 2, 72.8% with 4, 24.5% with 8.
- **Honest reading**: from 5 layers on, the plain GCN does not even fit its 140 training labels (54% at 5 layers, 24% at 8). It fails to *train*, not only to generalise. Over-smoothing is one cause; dropout before every layer and shrinking gradients are others. We did not separate them.
- **Skip connections** $H^{\ell} = \sigma(\hat{A}H^{\ell-1}W^{\ell}) + H^{\ell-1}$ keep each node's own signal. They repair training (69.5% test at 8 layers), but depth still does not *help* here.

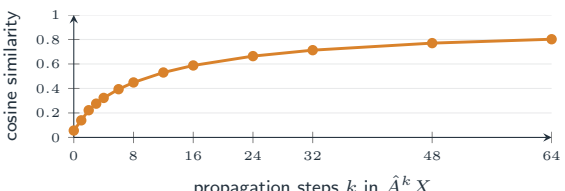
Over-smoothing [10,11]: every layer averages over neighbours. Average often enough and all nodes look alike.

Why: powers of $\hat{A}$

$\hat{A}$ is symmetric: $\hat{A} = \sum_i \lambda_i u_i u_i^{\top}$, with $1 = \lambda_1 > |\lambda_2| \geq \dots$ on a connected graph. Ignoring W and σ, k layers compute

$$\hat{A}^k X = \sum_i \lambda_i^k \, u_i \, (u_i^{\top} X) \xrightarrow{k \to \infty} u_1 \, (u_1^{\top} X), \qquad (u_1)_v \propto \sqrt{\tilde{d}_v}.$$

Every row becomes a multiple of the *same* vector: what a node knew about itself is gone, only its degree is left.



Measured on Cora: mean cosine similarity over 5,000 random node pairs.

Random pairs start near-orthogonal (0.06); 0.22 after 2 steps, 0.45 after 8, 0.80 after 64. A little smoothing makes same-topic papers alike and helps. A lot makes *all* papers alike.

Variants I: GraphSAGE and graph attention (GAT)

GraphSAGE [4]: mean, then concatenate

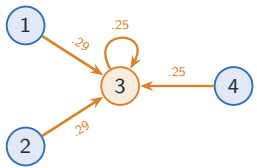
$$h'_v = \sigma\big(\big[\, h_v \;\|\; \bar{h}_{\mathcal{N}(v)} \big] W\big) = \sigma\big(h_v W_{\text{self}} + \bar{h}_{\mathcal{N}(v)} W_{\text{neigh}}\big)$$

- $\bar{h}_{\mathcal{N}(v)}$: mean of the neighbours' vectors; $\|$: concatenation. The node's own vector gets **its own weights**.
- For large graphs: *sample* a few neighbours per node, train on mini-batches. The rule also applies to nodes never seen in training (inductive).

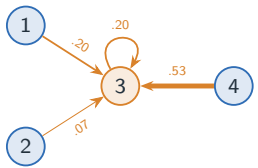
GAT [5]: learn the weight of each neighbour

$$e_{vu} = \mathrm{LeakyReLU}\big(a_{\text{dst}} \cdot (h_v W) + a_{\text{src}} \cdot (h_u W)\big)$$
$$\alpha_{vu} = \mathrm{softmax}_u(e_{vu}), \qquad h'_v = \sigma\big(\textstyle\sum_u \alpha_{vu}\, h_u W\big)$$

- u runs over $\mathcal{N}(v) \cup \{v\}$. GCN's fixed $1/\sqrt{\tilde{d}_v \tilde{d}_u}$ becomes an **attention coefficient** α_{vu} computed from the two nodes' features; W, a_{dst}, a_{src} are trained. Several **heads** run in parallel and are concatenated (8 heads of 8 units in [5] and here).



GCN: weights from degrees



GAT: weights from features

Node 3 of the example. Right: $W{=}I$, $a_{\text{dst}}{=}0$, $a_{\text{src}}{=}(1,0)$, scores $(1,0,1,2)$. Illustrative, not trained.

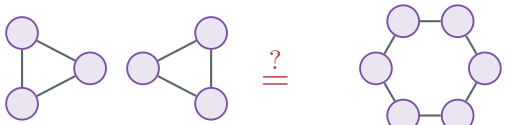
Cora, 2 layers	parameters	s / run	test accuracy
GCN (16 hidden units)	23,063	5.0	79.9 ± 1.3 %
GraphSAGE-mean (16)	46,103	4.6	78.3 ± 2.1 %
GAT (8×8)	92,373	9.9	79.7 ± 2.0 %

Measured, full-batch; 10 splits (GAT: first 5). See slide 19.

Remark: a transformer is a GNN on the complete graph

Self-attention [18]: $h'_v = \sum_u \alpha_{vu}\, h_u W$, $\alpha_{vu} = \mathrm{softmax}_u(q_v \cdot k_u / \sqrt{d})$ over *all* tokens u. Everyone is everyone's neighbour; order comes from positional encodings, not edges.

Variants II: GIN, and what message passing cannot see



G_1: two triangles $\qquad$ G_2: one hexagon

Weisfeiler–Leman (1-WL) test [7]: can two graphs be told apart? Give every node the same colour, then repeatedly recolour each node by its own colour and the *multiset* $\{\{\cdot\}\}$ (a set with repeats) of its neighbours' colours:

$$c_v \;\leftarrow\; \mathrm{hash}\big(c_v,\ \{\{\, c_u : u \in \mathcal{N}(v) \,\}\}\big).$$

Different final colour counts prove the graphs different. **This is message passing** with AGG = "the multiset" and UPDATE = hash. Hence:

Theorem (Xu et al. [6], Morris et al. [8])

A message-passing GNN can distinguish two graphs *at most* when 1-WL can. It reaches that bound if AGG and UPDATE are injective (lose no information).

G_1 **vs** G_2 **defeats 1-WL**: in both, every node has two neighbours, each with two neighbours, and so on. All nodes keep one colour, yet G_1 has 2 triangles and G_2 none.

GIN, the graph isomorphism network [6]

$$h'_v = \mathrm{MLP}\Big((1+\epsilon)\, h_v + \sum_{u \in \mathcal{N}(v)} h_u\Big)$$

Sum: it keeps what mean and max discard (slide 7), how *many* neighbours of each kind. **MLP**: one linear layer cannot represent every injective update. With both, GIN is exactly as discriminating as 1-WL.

Measured (`walkthrough.py`, part 8)

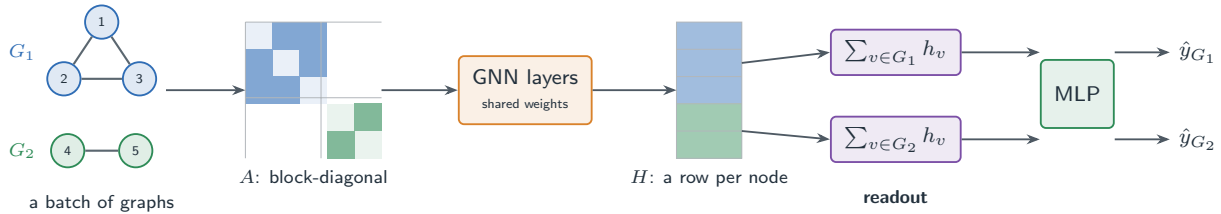
Random 3-layer GIN, all node features 1, sum readout:

G_1 (two triangles): $\quad (0.999,\ 0.932,\ 0.032,\ 1.432)$
G_2 (hexagon): $\quad (0.999,\ 0.932,\ 0.032,\ 1.432)$

Identical for *any* weights: this model cannot count triangles.

- **In practice** the limit bites when nodes lack distinguishing features. With rich features, as in Cora, it rarely matters: GCN is weaker than GIN in theory and does well there.
- Remedies: node ids, positional encodings, higher-order networks [8].

Graph-level prediction: readout and batching



Readout (global pooling): collapse a graph's node vectors into one,

$$h_G = \sum_{v \in G} h_v \qquad \text{or} \qquad h_G = \frac{1}{|G|} \sum_{v \in G} h_v,$$

then classify h_G with an MLP. A sum or mean ignores node order, so the whole model is permutation *invariant* (slide 6) and accepts graphs of any size. **Sum** keeps size information (number of atoms); **mean** is stable across sizes.

By hand (`walkthrough.py`, part 9):
$x = (1,0),(0,1),(1,1) \mid (2,0),(0,2)$. After one propagation the rows of $\hat{A}X$ are $(\frac{2}{3}, \frac{2}{3}) \times 3$ and $(1,1) \times 2$.

	triangle G_1	edge G_2	
sum readout	(2, 2)	(2, 2)	same
mean readout	(.667, .667)	(1, 1)	different

Batching trick: many graphs = one big graph

Renumber the nodes consecutively and merge the edge lists. A becomes **block-diagonal**: no edge joins two graphs, so no message crosses, and one sparse product processes the whole batch. `batch` $= (0,0,0,1,1)$ says which graph each node is in; the readout sums rows by that id (`batch_graphs`, `readout` in `gnn.py`).

- Here the data *is* i.i.d.: many separate graphs, split into train/val/test *by graph*.
- In the example the sum cannot separate G_1 from G_2 and the mean can. For one triangle vs two copies of it, the reverse. G_1 alone also gives $(2,2)$: batching changes nothing.

Training a GCN: one graph, one batch, a masked loss

The recipe of [2], as used here

1. **Forward**: $Z = \hat{A}\,\mathrm{ReLU}(\hat{A}XW^1)\,W^2$ for *all* n nodes at once.
2. **Loss**: cross-entropy on the 140 training nodes only. With two labelled nodes and $p_1[y_1] = .881$, $p_6[y_6] = .953$: $\mathcal{L} = -\frac{1}{2}(\ln .881 + \ln .953) = \mathbf{0.088}$.
3. **Backward**, then one **Adam** step [19], learning rate 0.01.
4. Validation loss: keep the best weights; stop after 50 epochs without gain, at most 200.

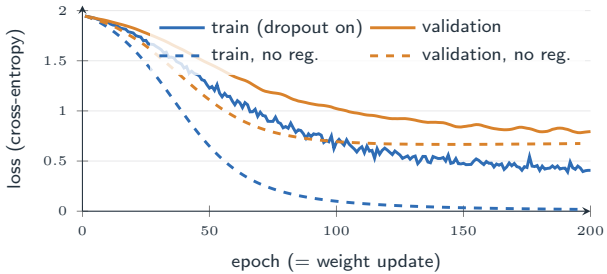
- **Full-batch**: the whole graph is one "batch": one epoch = one update. Very large graphs need neighbourhood sampling instead [4].
- **Regularisation**: **dropout** 0.5 on each layer's input, **weight decay** $5 \cdot 10^{-4}$.

Backpropagation through a GCN (`gcn_numpy.py`)

$$\delta^L = \tfrac{1}{|V_L|}\, M\,(P - Y) \qquad M\text{: 0/1 mask of labelled rows}$$

$$\partial\mathcal{L}/\partial W^\ell = (\hat{A}H^{\ell-1})^\top\,\delta^\ell$$

$$\delta^{\ell-1} = \left(\hat{A}^\top \delta^\ell\, W^{\ell\,\top}\right) \odot \sigma' \qquad \text{errors travel back along edges}$$

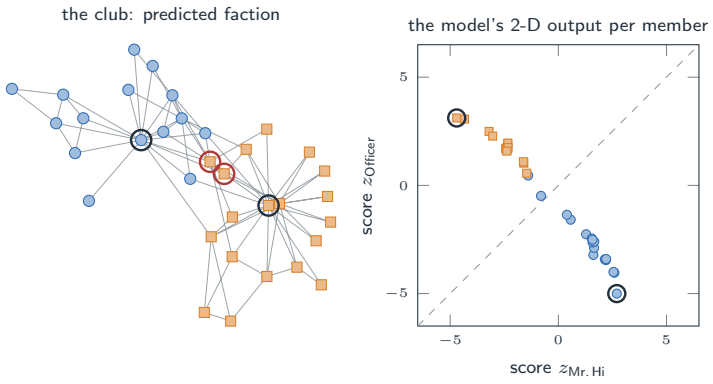


Measured: 2-layer GCN, Cora, split 0. Dashed: no regularisation.

GCN on Cora, 10 splits	test accuracy	epochs (split 0)
dropout 0.5 + weight decay	79.9 ± 1.3 %	200
no dropout, no weight decay	76.6 ± 1.6 %	196

- **Without regularisation** the training loss falls to 0.018: the 140 labels are memorised. Validation loss bottoms out at 0.67.
- **With it**, the validation loss is still falling at epoch 200 (0.78): the fixed budget of [2] ends the run before convergence.
- **Honest reading**: worth about 3 points over 10 splits; on split 0 alone only 79.7% vs 80.7%.

A graph you can see: Zachary's karate club from two labels



Measured, seed 0. Left: colour = *predicted* faction (blue circle: Mr. Hi, orange square: Officer); black rings: the 2 labelled nodes; red rings: the 2 errors (nodes 2, 8). Right: colour = *true* faction; dashed: decision boundary.

The data [16]: 34 members of a karate club and their 78 friendships. After a dispute the club split into the instructor's group ("Mr. Hi", node 0) and the president's ("Officer", node 33), 17 each. **The task**: given the friendships and the faction of the *two leaders only*, predict everyone else's. No node features: $X = I$.

Model: GCN $34 \to 16 \to 2$ (594 weights), no dropout, trained as on slide 15. Of the 32 other members, 8 are validation nodes and 24 test nodes.

test accuracy, 20 seeds	mean ± std
majority class	48.3 ± 6.5 %
MLP on $X = I$ (no graph)	49.6 ± 11.4 %
GCN	95.6 ± 3.2 %
label propagation [14]	**97.3 ± 2.0 %**

- **Two labels go a long way.** With one-hot ids as features all the information is in the edges: the MLP is at chance, the GCN at 95.6%.
- **Honest reading**: the GCN does *not* beat label propagation, a rule with no trainable weights. It is worse on 8 of 20 seeds and better on 0. (One test node is 4 points.)
- With no features and neighbours that share labels, spreading labels along edges is all there is to learn. A GNN earns its cost when *features* matter too (slide 19).

Code: the GCN layer from scratch, and in PyTorch

gcn_numpy.py: the normalised adjacency $\hat{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$ and the forward pass in NumPy (lines verbatim, some omitted)

```
def normalized_adjacency(A: np.ndarray, self_loops: bool = True, norm: str = "sym") -> np.ndarray:
    A_tilde = A + np.eye(len(A)) if self_loops else A.astype(float)
    d = A_tilde.sum(axis=1)                                  # degrees, self-loop included
    d_inv_sqrt = 1.0 / np.sqrt(np.maximum(d, 1e-12))
    return d_inv_sqrt[:, None] * A_tilde * d_inv_sqrt[None, :]   # entry (v,u) = 1/sqrt(d_v d_u)
    def forward(self, A_hat: np.ndarray, X: np.ndarray) -> np.ndarray:
        self.A_hat, self.H, self.S = A_hat, [X], []          # caches for the backward pass
        for l, (W, b) in enumerate(zip(self.W, self.b)):
            self.S.append(A_hat @ self.H[-1])                # aggregate: weighted sum over neighbours
            Z = self.S[-1] @ W + b                           # update: shared linear map
            self.H.append(np.maximum(Z, 0) if l < len(self.W) - 1 else Z)
        return self.H[-1]                                    # logits, one row per node
```

gnn.py: the same layer as a PyTorch module with a sparse $\hat{A}$; below it, the loop of `GNN.forward` that stacks layers

```
class GCNLayer(nn.Module):
    def __init__(self, n_in: int, n_out: int):
        self.lin = Linear(n_in, n_out, bias=False)        # W
        self.bias = nn.Parameter(torch.zeros(n_out))
    def forward(self, H: torch.Tensor, ops: dict) -> torch.Tensor:
        return torch.sparse.mm(ops["A_hat"], self.lin(H)) + self.bias     # A_hat (H W) + b
        for l, layer in enumerate(self.layers):
            H = layer(dropout(H, self.dropout, self.training), ops)
            if l < len(self.layers) - 1:
                H = F.elu(H) if self.kind == "gat" else F.relu(H)
```

`ops["A_hat"]`: built once from the edge list by `graph_operators`. `test_gnn.py`: both versions give the same loss and gradients to 10^{-10}.

Code: the training and testing harness

`train.py`, `fit()`: full-batch training, masked loss, early stopping (lines verbatim; logging omitted; best weights restored afterwards)

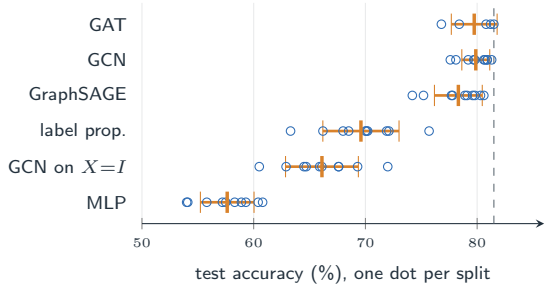
```python
def fit(model: GNN, d: dict, lr: float = 0.01, weight_decay: float = 5e-4, epochs: int = 200,
        patience: int = 50, verbose: bool = False) -> list[dict]:
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    train = d["masks"]["train"]
    for epoch in range(1, epochs + 1):
        model.train()
        logits = model(d["X"], d["ops"])                  # 1. forward pass: ALL nodes, all edges
        loss = F.cross_entropy(logits[train], d["y"][train])   # 2. loss: labelled nodes only
        opt.zero_grad()
        loss.backward()                                   # 3. backpropagation
        opt.step()                                        # 4. Adam update
        val = evaluate(model, d, "val")                   # no weight update from these nodes
        if val["loss"] < best_val - 1e-5:
            best_val, bad_epochs = val["loss"], 0
            best_state = copy.deepcopy(model.state_dict())
        else:
            bad_epochs += 1
            if patience and bad_epochs >= patience:       # early stopping
                break
```

`evaluate()`: the forward pass always uses the whole graph; the mask only selects which nodes are scored

```python
def evaluate(model: GNN, d: dict, split: str) -> dict:
    model.eval()                                          # switches dropout off
    logits, mask = model(d["X"], d["ops"]), d["masks"][split]
    pred = logits.argmax(dim=1)
    return dict(loss=F.cross_entropy(logits[mask], d["y"][mask]).item(),
                accuracy=(pred[mask] == d["y"][mask]).float().mean().item(), pred=pred)
```

`python train.py --model gcn|mlp|sage|gat|gin --layers 2 --hidden 16 --seed 0` prints test accuracy, two baselines and a confusion matrix.

Results on Cora: features, graph, or both?



Orange: mean ± 1 std. Dashed: 81.5%, GCN in [2] on the public split.

model (2 layers)	feat.	graph	weights	test accuracy
majority class			0	10.5 ± 0.8 %
MLP	✓		23,063	57.6 ± 2.4 %
GCN on $X = I$		✓	43,463	66.1 ± 3.3 %
label propagation [14]		✓	0	69.6 ± 3.4 %
GraphSAGE-mean	✓	✓	46,103	78.3 ± 2.1 %
GCN	✓	✓	23,063	**79.9 ± 1.3 %**
GAT (5 splits)	✓	✓	92,373	79.7 ± 2.0 %

Setup. Cora [15]: 2,708 papers, 5,278 undirected citation links, 1,433 binary word features (about 18 per paper), 7 topics. 10 random splits 140 / 500 / 1000 (slide 5); the seed also sets the initial weights. Mean ± std over splits; settings of slide 15. `python train.py` reproduces split 0: 80.7%.

- **Neither source is enough alone.** Words without the graph: 57.6%. Graph without words: 69.6% (label propagation).
- **Together**: 79.9%. The GCN has the same 23,063 weights as the MLP; the only difference is the factor $\hat{A}$ in each layer. It wins on 10 of 10 splits, by 22 points on average.
- **The variants do not separate.** GraphSAGE, GCN and GAT are within one standard deviation. GCN beats GraphSAGE on 7 of 10 splits; GAT beats GCN on 2 of 5. None was tuned.
- **Against the literature**: our GCN ranges from 77.6% to 81.3%; [2] reports 81.5% and [5] 83.0% for GAT, on the public split, which we do *not* use. Splits change the numbers [13]: a loose comparison, about 1.5 points lower.
- **Caveat**: 81% of Cora's links join same-topic papers. Where neighbours tend to differ, neighbourhood averaging can hurt.

References and how to run the code

[1] Scarselli, Gori, Tsoi, Hagenbuchner, Monfardini. *The Graph Neural Network Model.* IEEE Trans. Neural Networks 20(1), 2009.

[2] Kipf, Welling. *Semi-Supervised Classification with Graph Convolutional Networks.* ICLR 2017. arXiv:1609.02907

[3] Gilmer, Schoenholz, Riley, Vinyals, Dahl. *Neural Message Passing for Quantum Chemistry.* ICML 2017. arXiv:1704.01212

[4] Hamilton, Ying, Leskovec. *Inductive Representation Learning on Large Graphs.* NeurIPS 2017. arXiv:1706.02216

[5] Veličković, Cucurull, Casanova, Romero, Liò, Bengio. *Graph Attention Networks.* ICLR 2018. arXiv:1710.10903

[6] Xu, Hu, Leskovec, Jegelka. *How Powerful are Graph Neural Networks?* ICLR 2019. arXiv:1810.00826

[7] Weisfeiler, Leman. *The Reduction of a Graph to Canonical Form and the Algebra which Appears Therein.* Nauchno-Technicheskaya Informatsia 2(9), 1968.

[8] Morris, Ritzert, Fey, Hamilton, Lenssen, Rattan, Grohe. *Weisfeiler and Leman Go Neural: Higher-order Graph Neural Networks.* AAAI 2019. arXiv:1810.02244

[9] Zaheer, Kottur, Ravanbakhsh, Poczos, Salakhutdinov, Smola. *Deep Sets.* NeurIPS 2017. arXiv:1703.06114

[10] Li, Han, Wu. *Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning.* AAAI 2018. arXiv:1801.07606

[11] Oono, Suzuki. *Graph Neural Networks Exponentially Lose Expressive Power for Node Classification.* ICLR 2020. arXiv:1905.10947

[12] Yang, Cohen, Salakhutdinov. *Revisiting Semi-Supervised Learning with Graph Embeddings.* ICML 2016. arXiv:1603.08861

[13] Shchur, Mumme, Bojchevski, Günnemann. *Pitfalls of Graph Neural Network Evaluation.* arXiv:1811.05868, 2018.

[14] Zhou, Bousquet, Lal, Weston, Schölkopf. *Learning with Local and Global Consistency.* NIPS 2003.

[15] Sen, Namata, Bilgic, Getoor, Gallagher, Eliassi-Rad. *Collective Classification in Network Data.* AI Magazine 29(3), 2008. Cora files from github.com/tkipf/pygcn.

[16] Zachary. *An Information Flow Model for Conflict and Fission in Small Groups.* Journal of Anthropological Research 33(4), 1977.

[17] Bronstein, Bruna, Cohen, Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges.* arXiv:2104.13478, 2021.

[18] Vaswani et al. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762

[19] Kingma, Ba. *Adam: A Method for Stochastic Optimization.* ICLR 2015. arXiv:1412.6980

Run it (Python, NumPy, PyTorch; no graph library needed)

`python test_gnn.py`	seven checks, incl. gradient check
`python walkthrough.py`	the hand calculations (slides 3, 6–15)
`python gcn_numpy.py`	from-scratch GCN learns the karate club
`python train.py`	the harness on Cora; `--help` lists the knobs
`python experiments.py`	all figures and numbers (about 8 min)

Exercises. Remove the self-loops and retrain. Replace $\hat{A}$ by the mean $\tilde{D}^{-1}\tilde{A}$. Train with 5 labels per class. Make the 8-layer GCN train.